Introduction to Bash Scripting

Automation and the Command Line

Bernal Manzanilla Saavedra

Concordia University – IT110

March 27, 2026

The Environment: Where are we coding?

Today, you have two options for writing and executing your scripts:

1. **Native Linux / WSL:**
   - You will write your code in a text editor (like `nano`).
   - You will save the file with a `.sh` extension (e.g., `script.sh`).
   - You will execute it in the terminal by typing `bash script.sh`.

2. **Google Colab:**
   - Colab usually runs Python. To force it to run a Bash script, you must type the magic command `%%bash` at the very top of your cell.
   - When you click the "Play" button, the whole cell runs as a script.

Basic I/O and Redirection

The `echo` command prints text to the Standard Output (your screen). However, we can intercept that text and send it to a file.

```
echo "Log started." > system_log.txt
echo "Operation successful." >> system_log.txt
```

Line-by-Line Breakdown:

- **The Single Bracket (>):** This takes "Log started." and creates the text file. **Warning:** If the file already exists, > will completely erase and overwrite it.
- **The Double Bracket (>>):** This is much safer. It takes "Operation successful." and appends it to the very bottom of the existing file without destroying old data.

Variable Catching and Pipes

Scripts are powerful because they can capture dynamic data and pass it around.

```
current_time=$(date)
echo "The script was run on: $current_time"
ls -l | grep ".txt"
```

Line-by-Line Breakdown:

- **Variable Catching `$()`:** The computer runs the `date` command first. Instead of printing it to the screen, the `$()` captures the output and locks it inside the variable named `current_time`.
- **The Pipe `|` and `grep`:** We run `ls -l` to list all files. The pipe (`|`) catches that massive list and feeds it directly into `grep`. `grep` then searches the list and only prints the lines that contain ".txt".

Loops and The Black Hole

If we want to automate a repetitive task, we use loops. If we want to hide messy error messages, we use `/dev/null`.

```
for i in 1 2 3
do
  echo "Processing item $i..."
  ls /fake_directory 2> /dev/null
done
```

Line-by-Line Breakdown:

- **The Loop:** The script will run the commands between `do` and `done` exactly three times. The variable `$i` changes to 1, then 2, then 3.
- **The Black Hole:** We are trying to list a directory that doesn't exist, which generates an ugly error. By redirecting the error output (2>) to `/dev/null`, the error is silently destroyed and the user's screen stays clean.

While Loops

A `for` loop runs a specific number of times. A `while` loop runs indefinitely until a condition is met.

```
counter=1
while [ $counter -le 2 ]
do
  echo "While loop count: $counter"
  ((counter++))
done
```

Line-by-Line Breakdown:

- **The Condition:** `-le 2` means "less than or equal to 2". The loop keeps going as long as this is true.
- **The Increment:** `((counter++))` adds 1 to the counter every loop. If you forget this line, the counter stays at 1 forever, causing an **infinite loop** that will crash your program.

Time to Script!

Open your Terminal or Google Colab and follow the handout.